

Prime-GPF Magmas

Mathematics, Lean Verification, and Literature Context

Revised Theorem Compendium

Project consolidation for Anders Hellström

Revised 13 September 2026

AI-ASSISTED REPORT — READ THIS NOTICE

AI and deep-research tools assisted with literature discovery, source comparison, drafting, and XeLaTeX production. Mathematical claims and bibliographic entries still require human review. Lean compiler/kernel acceptance is a separate machine-checking process: it is evidence about formal proof terms relative to formal statements and imports, and must not be conflated with AI validation. A complete disclosure appears in [Chapter C](#).

Abstract and verification snapshot

This report revises and expands the June 2026 *Theorem Compendium for Prime GPF Magmas* [Hel26]. The subject is a family of binary operations on the prime numbers obtained by taking the greatest prime factor of three elementary kernels,

$$p \boxplus q = P^+(p + q + 1), \quad p \boxtimes q = P^+(pq + 1), \quad p \uparrow_G q = P^+(p^q + 1).$$

The revision has three purposes. First, it makes the mathematical exposition self-contained by explaining the algebraic, analytic, cyclotomic, dynamical, and formal-verification concepts that the theorems use. Second, it reconciles every numbered result with the Lean 4 formalization. Third, it places the construction in its relevant literature and reports a targeted prior-art/novelty search.

Formal verification snapshot as of 13 September 2026

The author-supplied local build and audit record reports **33 complete unconditional compiler-verified numbered results** and **6 compiler-verified conditional numbered results**, with no statement-only and no false *numbered* result in the corrected inventory. The conditional results are 6.1 and 6.3, both depending only on `PrimeAPIInput`, and 8.3, 8.4, 8.5, and 9.3, all depending only on `ZsigmondyInput`. The historical, originally printed form of 5.5 is retained separately and formally refuted; the official corrected 5.5 adds the necessary condition $r \neq 2$ and is unconditionally verified. The latest audit shown to the author contains only the ordinary dependencies `propext`, `Classical.choice`, and `Quot.sound`, with no `sorryAx` or compiler-trust shortcuts.

The formal development targets Lean 4.19.0 and a pinned mathlib revision. A successful Lean build establishes that the kernel has accepted proof terms for the stated declarations relative to their imported definitions and axioms; Lean’s own documentation emphasizes both this guarantee and the separate obligation to check that the formal statement means what the informal author intends [Leaa; Leab].

Contents

Abstract and verification snapshot	ii
1. Scope, source record, and formal verification	1
1.1. From a short compendium to a checked research record	1
1.2. What “verified” means here	1
1.3. Repository and reproducibility record	1
1.4. Source hierarchy used in this revision	2
2. Greatest prime factors, magmas, and the Lean model	3
2.1. Greatest prime factor	3
2.2. Three prime-GPF operations	3
2.3. Fibers, translations, diagonals, and orbits	3
2.4. Smooth numbers	4
2.5. Congruence rings and multiplicative order	4
3. Basic algebraic laws	5
4. Exact fibers and smoothness	7
4.1. The general fiber theorem	7
4.2. Small exact fibers	7
5. Congruences, multiplicative orders, and cyclotomic forcing	8
5.1. Modular inverse structure	8
5.2. Diagonal multiplication and order four	8
5.3. Quadratic character	8
5.4. The correction to Theorem 5.5	8
6. Arithmetic progressions, smooth counting, and density	10
6.1. Prime number theorem in arithmetic progressions	10
6.2. An elementary smooth-number bound strong enough for the fibers	11
6.3. Zero density: the new conditional formal proof	11
7. Prime dynamics	13
7.1. Fixed points of additive translations	13
8. Exponential GPF structure and the Zsigmondy frontier	15
8.1. Unconditional elementary structure	15
8.2. Primitive divisors and Bang–Zsigmondy	15
9. Mixed-operation collisions	17
10. Lean formalization and proof-status map	19
10.1. Architecture of the source	19
10.2. Complete numbered-status table	20
10.3. What formalization clarified	21
11. Related work and novelty assessment	22
11.1. Search methodology	22
11.2. Direct prior art on greatest-prime-factor magmas	22
11.3. Comparison with the three kernels in this report	22

11.4. Classical ingredients versus project-specific synthesis	23
11.5. Formal-library search	23
12. Conclusions and open problems	24
12.1. Immediate formal targets	24
12.2. Mathematical directions	24
12.3. Interpretive conclusion	24
A. Computational appendix: PARI/GP	25
B. Lean declaration guide	26
C. Complete AI-use disclosure	27
D. Change log and author-review checklist	28
Bibliography	29

1. Scope, source record, and formal verification

1.1. From a short compendium to a checked research record

The original 15-page report was designed as a deduplicated list of the main prime-GPF results. It contained concise proofs, a computational appendix, and a bibliography, but it predated the completed Lean work for Theorem 6.2, the conditional Lean proof of Theorem 6.3, and the correction of Theorem 5.5. The present version therefore treats the old document as a historical baseline rather than an authority whenever it conflicts with the checked formal source.

The formal project is organized around a declarative inventory of 39 numbered propositions (3.1–9.8) and proof modules that discharge them. In Lean, a declaration such as

```
def t6_3 : Prop := ...
```

merely gives a name to a proposition. It does *not* prove it. A declaration such as

```
theorem proof_6_3_from_PNT_AP (H : PrimeAPIInput) : Claims.t6_3 := by
  ...
```

proves an implication: if a term of type `PrimeAPIInput` is supplied, Lean checks a term of type `Claims.t6_3`. This is a genuine formal proof of the implication, but not a proof of the external input.

1.2. What “verified” means here

Lean elaboration, proof terms, and the kernel

Lean translates notation, tactics, coercions, and typeclass search into a proof term in its dependent type theory. The kernel then checks that proof term against the claimed type. The Lean documentation explicitly separates the validity of a formal proof from the semantic question of whether the formal theorem statement faithfully expresses the intended mathematics [Leab]. Accordingly, this report uses *compiler-verified* to mean that the checked local project accepted the declaration, not that an AI system or informal argument endorsed it.

The project further audits axiom dependencies. The accepted foundational items in the supplied audit are `propext`, `Classical.choice`, and `Quot.sound`. The project policy forbids using `sorry`, `admit`, a newly postulated mathematical axiom, `native_decide`, `Lean.ofReduceBool`, or `Lean.trustCompiler` to bypass a missing proof.

1.3. Repository and reproducibility record

The Lean source is maintained in the private GitHub repository `AndersH3/PrimeGPF`; the associated local working directory reported during verification was

```
~/Documents/prime_gpf_lean_new/prime_gpf_lean.
```

At the time of this report, the remote repository and the local tree were temporarily asynchronous for some of the newest density work: the author’s local transcript is therefore the source of truth for the verified 6.3 status. This distinction matters in a reproducibility report and should be removed once the complete verified local milestone is pushed.

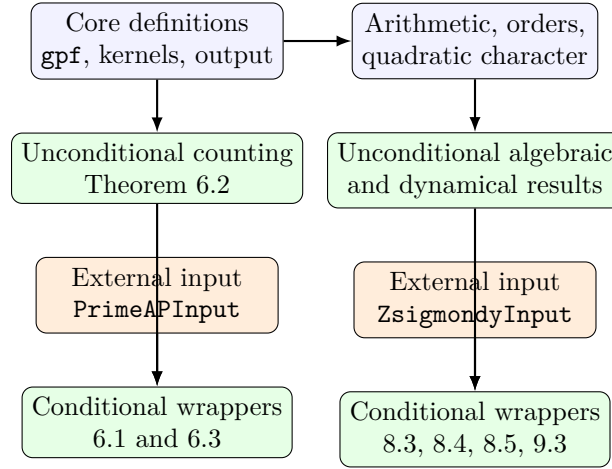


Figure 1.1.: Logical architecture of the formalization. Green boxes represent checked theorem layers; orange boxes are the two remaining external mathematical inputs.

1.4. Source hierarchy used in this revision

When sources disagree, this report uses the following order of authority: (1) the checked Lean statement and current verification transcript for formal status; (2) the original compendium for historical exposition and notation; (3) standard mathematical references for classical ingredients; and (4) novelty-search results for contextual comparison. This prevents a common error in formalization reports: silently upgrading an informal proof sketch into a machine-verified claim.

2. Greatest prime factors, magmas, and the Lean model

2.1. Greatest prime factor

For an integer $n > 1$, write

$$P^+(n) = \max\{\ell \in \mathbb{P} : \ell \mid n\}.$$

This is the greatest (largest) prime factor of n , a standard arithmetic function; related sequence data are catalogued in OEIS A006530 and standard references [OEI; Wei]. The formal project intentionally defines its own finite search `gpf` rather than depending on a version-sensitive library API. Its core specification proves that, for $n > 1$, `gpf n` is prime, divides n , and dominates every prime divisor of n .

Why the kernels are always in the domain of P^+

For primes $p, q \geq 2$, each kernel $p + q + 1$, $pq + 1$, and $p^q + 1$ exceeds 1. Hence it has a prime divisor and therefore a greatest prime divisor. Lean formalizes this elementary fact as `kernel_gt_one`, which is the gateway from the total computational `gpf` function to the mathematically intended operation.

2.2. Three prime-GPF operations

Let $\mathbb{P}$ denote the set of primes. Define

$$\begin{aligned} K_+(p, q) &= p + q + 1, & p \boxplus q &= P^+(K_+(p, q)), \\ K_\times(p, q) &= pq + 1, & p \boxtimes q &= P^+(K_\times(p, q)), \\ K_{\text{exp}}(p, q) &= p^q + 1, & p \uparrow_G q &= P^+(K_{\text{exp}}(p, q)). \end{aligned}$$

The symbols are notation; the structural fact is that all three outputs are primes.

Definition 2.1 (Magma). A *magma* is a set B equipped with a closed binary operation $B \times B \rightarrow B$. No associativity, identity, commutativity, or cancellation is assumed [BS81]. Thus showing closure is enough to conclude that $(\mathbb{P}, \boxplus)$, $(\mathbb{P}, \boxtimes)$, and $(\mathbb{P}, \uparrow_G)$ are magmas.

2.3. Fibers, translations, diagonals, and orbits

For a binary operation $\star$ and fixed a , the *left translation* is $L_a(q) = a \star q$. A *fiber* over r is the set of inputs mapping to r . The *diagonal map* is $D(p) = p \star p$. Iterating a unary map T produces an orbit $x_0 = x$, $x_{n+1} = T(x_n)$. An orbit is periodic if $x_d = x_0$ for some $d > 0$, and eventually periodic if repetition begins after some transient stage.

Lean uses the subtype

```
abbrev Prime := {p : // Nat.Prime p}
```

so that a value of type `Prime` carries both a natural number and a proof of primality. The functions `operate`, `leftTranslation`, `diagonal`, and `orbit` implement these ideas directly.

2.4. Smooth numbers

Definition 2.2 (*r*-smooth). A positive integer s is *r*-smooth if every prime divisor of s is at most r . In modern analytic number theory one often writes $P^+(s) \leq r$. Smooth numbers are a classical subject with asymptotic theories due to Dickman, de Bruijn, Hildebrand, Tenenbaum, and others [Dic30; Bru51; HT86; HT93].

For this project, the relevant regime is unusually elementary: r is fixed while the size bound x grows. Then an *r*-smooth number is built from only finitely many primes, and a simple exponent-box count already yields a polylogarithmic bound. The project does not need the full Dickman–de Bruijn asymptotic machinery to prove Theorem 6.2.

2.5. Congruence rings and multiplicative order

The type `ZModr` represents integers modulo r . When r is prime it is a finite field. The *multiplicative order* of a nonzero residue $a \pmod{r}$ is the least positive k with $a^k \equiv 1 \pmod{r}$. In a finite field this order divides $r - 1$, the size of the multiplicative group. The project bridges its explicit predicate `ExactOrder` to mathlib’s group-theoretic `orderOf` in `Orders.lean`.

Definition 2.3 (Primitive divisor in this project). `PrimitiveDivisorrp` says that r is prime, $p^k \equiv 1 \pmod{r}$, and no exponent $0 < j < k$ has $p^j \equiv 1 \pmod{r}$. For $k > 0$ this is exactly an order statement. The positivity caveat matters: the original generic assertion “a primitive divisor of order k implies $k \mid r - 1$ ” was false at $k = 0$, and Lean forced that hidden hypothesis to be made explicit.

3. Basic algebraic laws

This chapter corresponds to results 3.1–3.8. All are unconditional and compiler-verified in the current inventory.

Theorem 3.1 (3.1: closure and divisibility certificates). *For all primes p, q and each operation $\star \in \{\boxplus, \boxtimes, \uparrow_G\}$, the output $p \star q$ is prime and divides the corresponding kernel.*

Idea. This is immediate from the greatest-prime-factor specification. In Lean the proof is `proof_3_1`, built from `output_spec`. $\square$

Theorem 3.2 (3.2: commutativity split). *The operations $\boxplus$ and $\boxtimes$ are commutative, whereas $\uparrow_G$ is not.*

The first two statements reflect symmetry of $p + q + 1$ and $pq + 1$. For the exponential operation, a concrete witness suffices: $2 \uparrow_G 3 = P^+(9) = 3$, while $3 \uparrow_G 2 = P^+(10) = 5$.

Theorem 3.3 (3.3: nonassociativity). *None of $\boxplus$, $\boxtimes$, or $\uparrow_G$ is associative.*

Lean verifies explicit small counterexamples. This is an instructive use of formal computation: a universal algebraic law is disproved by a single checked instance.

Theorem 3.4 (3.4: anti-projection, diagonal non-idempotence, and no identities). *For primes p, q , $p \boxtimes q \notin \{p, q\}$; $p \boxplus p \neq p$; and $p \uparrow_G q \neq p$. Consequently none of the three magmas has a two-sided identity.*

For multiplication, $pq + 1 \equiv 1 \pmod{p}$ and $\equiv 1 \pmod{q}$, so neither input can divide the kernel. The other claims follow from equally direct congruences.

Theorem 3.5 (3.5: odd-prime closure split). *Let $\mathbb{P}_{\text{odd}} = \mathbb{P} \setminus \{2\}$. Then $\mathbb{P}_{\text{odd}}$ is closed under $\boxplus$ and $\uparrow_G$, but not under $\boxtimes$.*

For $\boxplus$, the kernel of two odd primes is odd. For $\uparrow_G$, the project gives a direct factorization/odd-cofactor proof, independent of Zsigmondy’s theorem. Multiplication fails: $3 \boxtimes 5 = P^+(16) = 2$.

Theorem 3.6 (3.6: cancellation and distributive laws fail). *The operations $\boxplus$ and $\boxtimes$ are not left-cancellative. Moreover $\boxtimes$ does not distribute over $\boxplus$, and $\boxplus$ does not distribute over $\boxtimes$.*

Proposition 3.7 (3.7: mixed associativity failures). *The three mixed laws tracked in the source inventory all fail; the Lean proof uses the common witness $(2, 2, 2)$ after unfolding the operations.*

Theorem 3.8 (3.8: monotonicity fails in both coordinates). *With the usual ordering of primes, none of the three operations is monotone in either coordinate.*

This is a conceptual warning: the kernels themselves grow monotonically in several variables, but taking the *greatest prime factor* destroys monotonicity. A larger integer can have a smaller largest prime divisor.

Lean correspondence for this chapter

Results 3.1–3.4 and 3.6–3.8 are implemented in `PrimeGPF/Elementary.lean`. Result 3.5 has a direct unconditional proof in `PrimeGPF/DirectExponential.lean`. The audit classifies all eight numbered results as complete and compiler-verified.

4. Exact fibers and smoothness

4.1. The general fiber theorem

Theorem 4.1 (4.1: exact fixed-output criterion). *Let p, q, r be primes and let K be any of the three kernels. Then*

$$P^+(K(p, q)) = r \iff \exists s \geq 1 : K(p, q) = rs \text{ and } s \text{ is } r\text{-smooth.}$$

The statement isolates the key structure used throughout the report. Once r is declared to be the greatest prime factor, dividing by r leaves a cofactor whose prime factors cannot exceed r . Conversely, if the cofactor is r -smooth and r itself is prime, no prime larger than r can divide the kernel.

In Lean this appears first as the general lemma `gpf_fiber`, then as the numbered wrapper `proof_4_1`. It correctly allows the cofactor $s = 1$.

Theorem 4.2 (4.2: prime kernels and output two). *For each operation, the output equals its kernel exactly when the kernel is prime. The output is 2 exactly when the kernel is a positive power of 2.*

The second clause follows because an integer > 1 has greatest prime factor 2 precisely when all its prime factors are 2.

4.2. Small exact fibers

Theorem 4.3 (4.3: explicit fibers at 2 and 3). *The Lean theorem packages six classifications:*

- (a) $p \boxplus q = 2$ iff exactly one input is 2 and the other satisfies $q + 3 = 2^m$ (or symmetrically $p + 3 = 2^m$) for some $m \geq 3$.
- (b) $p \boxtimes q = 2$ iff $pq + 1 = 2^m$ for some $m > 0$.
- (c) If $p \boxtimes q = 2$, then p, q are odd and lie in opposite residue classes modulo 4.
- (d) For odd primes p, q , $p \boxplus q = 3$ iff $p + q + 1 = 3^m$ for some $m \geq 2$.
- (e) If $p, q > 3$ and $p \boxplus q = 3$, then $p \equiv q \equiv 1 \pmod{6}$.
- (f) $2 \boxtimes q = 3$ iff $2q + 1 = 3^m$ for some $m \geq 2$.

These are exact, not heuristic, fiber descriptions. They illustrate how an output restriction on P^+ often becomes a pure-power equation plus elementary congruences.

5. Congruences, multiplicative orders, and cyclotomic forcing

5.1. Modular inverse structure

Theorem 5.1 (5.1: modular inverse and affine laws). *For primes p, q , let $r = p \boxtimes q$ and $s = p \boxplus q$. Then*

$$pq \equiv -1 \pmod{r}, \quad p + q \equiv -1 \pmod{s}.$$

Moreover, since r cannot equal p or q , the residues of p and q modulo r are invertible and

$$q \equiv -p^{-1} \pmod{r}, \quad p \equiv -q^{-1} \pmod{r}.$$

The Lean statement avoids depending on notation for inverses by producing explicit inverse witnesses. This is a good example of a formal statement being slightly more implementation-stable than its conventional mathematical rendering.

5.2. Diagonal multiplication and order four

Theorem 5.2 (5.2: diagonal congruence and primitive-divisor form). *For every prime p ,*

$$p \boxtimes p \equiv 1 \pmod{4}.$$

If $r = p \boxtimes p$, then $p^2 \equiv -1 \pmod{r}$, so p has exact order 4 modulo r ; equivalently, r is a primitive prime divisor of $p^4 - 1$ in the project's positive-order sense.

The bridge from exact modular order to group-theoretic `orderOf` is formalized in `Orders.lean`.

Corollary 5.3 (5.3: diagonal orbit confinement). *After one iteration of the diagonal $\boxtimes$ map, every value is congruent to 1 (mod 4). Hence a periodic starting point must itself be 1 (mod 4).*

5.3. Quadratic character

Legendre symbol and quadratic character

For an odd prime r , the Legendre symbol (a/r) is 0 when $r \mid a$, 1 when a is a nonzero square modulo r , and -1 otherwise. It is multiplicative, and Euler's criterion gives $(a/r) \equiv a^{(r-1)/2} \pmod{r}$. Quadratic reciprocity relates (p/r) and (r/p) ; standard references include Ireland–Rosen [IR90]. The Lean development defines a finite `quadraticCharacter` explicitly and proves compatibility with `mathlib`'s character machinery.

Theorem 5.4 (5.4: quadratic-character constraint). *Let $r = p \boxtimes q$ and assume $r \neq 2$. Then*

$$\chi_r(p)\chi_r(q) = \chi_r(-1).$$

Thus if $r \equiv 1 \pmod{4}$, p and q have the same quadratic character modulo r ; if $r \equiv 3 \pmod{4}$, exactly one is a quadratic residue.

5.4. The correction to Theorem 5.5

Correction discovered and certified during formalization

The June 2026 report stated Theorem 5.5 without excluding $r = 2$. That statement is false. The Lean project contains a checked counterexample at $p = 3$, $q = 5$, $m = 1$: here $r = 3 \boxtimes 5 = 2$, the displayed congruence hypothesis holds, but the conclusion $r \equiv 1 \pmod{4}$ fails. The old proposition is retained as `t5_5_original` and formally refuted. The official numbered theorem is now the corrected version below.

Theorem 5.5 (5.5: corrected dyadic cyclotomic forcing). *Let p, q be primes and $m \geq 1$. Put $r = p \boxtimes q$. Assume*

$$r \neq 2, \quad q \equiv p^{2^m-1} \pmod{r}.$$

Then

$$\text{ord}_r(p) = 2^{m+1}, \quad r \equiv 1 \pmod{2^{m+1}},$$

r is a primitive divisor for order 2^{m+1} , and

$$r \mid p^{2^m} + 1 = \Phi_{2^{m+1}}(p).$$

The proof uses $r \mid pq + 1$ together with the assumed congruence to obtain $p^{2^m} \equiv -1 \pmod{r}$. Since $r \neq 2$, $-1 \neq 1$ in $\mathbb{Z}/r\mathbb{Z}$. Squaring gives exponent 2^{m+1} , while the halfway exponent is not 1, forcing exact order 2^{m+1} . The order divides $r - 1$, yielding the congruence. For 2-power cyclotomic indices, $\Phi_{2^{m+1}}(X) = X^{2^m} + 1$; see standard cyclotomic references [Was97].

Formal status of corrected 5.5

The official declaration `Claims.t5_5` is definitionally the corrected proposition and `proof_5_5` is compiler-verified. Its underlying proof is `proof_5_5_corrected` in `Orders.lean`; the wrapper lives in `FiveFive.lean`. The audit supplied with the project reports only `propext`, `Classical.choice`, and `Quot.sound`.

Theorem 5.6 (5.6: parity split for odd multiplicative inputs). *For odd primes p, q , equality $p \equiv q \pmod{4}$ prevents $p \boxtimes q = 2$; conversely, output 2 forces different residues modulo 4.*

6. Arithmetic progressions, smooth counting, and density

This is the chapter most substantially changed by the Lean development.

6.1. Prime number theorem in arithmetic progressions

Dirichlet versus PNT-AP

Dirichlet's theorem says that every reduced residue class modulo r contains infinitely many primes. The prime number theorem for arithmetic progressions is quantitatively much stronger: for $(c, r) = 1$,

$$\pi(x; r, c) \sim \frac{\text{Li}(x)}{\varphi(r)} \sim \frac{x}{\varphi(r) \log x}.$$

For prime r and $0 < c < r$, $\varphi(r) = r - 1$. Davenport gives a classical treatment of primes in arithmetic progressions [Dav00]. Current mathlib's `PrimesInAP` file proves Dirichlet infinitude, with main declarations giving infinitely many primes in an invertible residue class, but the searches performed for this report did not locate the quantitative PNT-AP ratio required by PrimeGPF [mat].

Definition 6.1 (The formal external input `PrimeAPInput`). For a prime modulus $r \neq 2$ and $0 < c < r$, let `apCountrcx` count primes $q \leq x$ satisfying $q \equiv c \pmod{r}$. The proposition `APAsymptoticrc` says, using Lean's filter limit notation,

$$\frac{\text{apCount}(r, c, x)}{x / ((r - 1) \log x)} \rightarrow 1 \quad (x \rightarrow \infty).$$

Then `PrimeAPInput` asserts `APAsymptoticrc` for every such r, c .

What `Filter.Tendsto` means

Lean represents limits using filters. `Filter.atTop` encodes the phrase “for sufficiently large x ”, and `nhdsL` is the filter of neighborhoods of L . Thus `Filter.TendstoFilter.atTop(nhdsL)` is the formal counterpart of $f(x) \rightarrow L$ as $x \rightarrow \infty$. This abstraction makes the same machinery usable for sequences, real-variable limits, and more general topological convergence.

Theorem 6.1 (6.1: progression reduction and conditional PNT-AP). *Fix primes a, r with $r \neq 2$.*

- (a) *If $a \neq r$, the primes q for which $r \mid aq + 1$ lie in exactly one nonzero residue class $q \equiv c \pmod{r}$; their exact finite count agrees with `apCountrcx`. Under `PrimeAPInput`, this count has the PNT-AP asymptotic.*
- (b) *If $(a + 1) \not\equiv 0 \pmod{r}$, the primes q for which $r \mid a + q + 1$ likewise lie in one reduced class, with the same asymptotic input.*
- (c) *If $(a + 1) \equiv 0 \pmod{r}$, then for prime q the additive divisibility condition is equivalent to $q = r$.*

The residue-class algebra and exact counting identities are unconditional in `Progressions.lean`. Only the asymptotic ratio is external.

6.2. An elementary smooth-number bound strong enough for the fibers

Let

$$S_r(x) = \#\{s \leq x : s \text{ is } r\text{-smooth}\}.$$

Every r -smooth integer has a unique factorization

$$s = \prod_{\substack{\ell \leq r \\ \ell \text{ prime}}} \ell^{e_\ell}.$$

If $s \leq x$, then $e_\ell \log \ell \leq \log x$, hence

$$0 \leq e_\ell \leq \left\lfloor \frac{\log x}{\log \ell} \right\rfloor.$$

Therefore the exponent vector belongs to a finite box and

$$S_r(x) \leq \prod_{\substack{\ell \leq r \\ \ell \text{ prime}}} \left(1 + \left\lfloor \frac{\log x}{\log \ell} \right\rfloor\right).$$

For fixed r , there are exactly $\pi(r)$ factors, so elementary comparison yields

$$S_r(x) = O_r((\log x)^{\pi(r)}).$$

This is much weaker than the refined smooth-number theory of Dickman–de Bruijn–Hildebrand–Tenenbaum, but it is perfectly adapted to a fixed smoothness bound r [Dic30; Bru51; HT93].

Theorem 6.2 (6.2: unconditional polylogarithmic fibers). *For every prime r , there is a constant $C_r > 0$ such that for every prime anchor a there is a threshold $N_{a,r}$ with, for $x \geq N_{a,r}$,*

$$F_{a,r}^{(+)}(x), F_{a,r}^{(\times)}(x) \leq C_r (\log x)^{\pi(r)}.$$

The same development proves the explicit smooth exponent-box bound above and a corresponding bound for `smoothCount`.

Lean proof architecture. If $a \boxplus q = r$, Theorem 4.1 writes $a + q + 1 = rs$ with s r -smooth. Likewise $a \boxtimes q = r$ gives $aq + 1 = rs$. For a fixed anchor, distinct q give distinct kernels and hence distinct cofactors. The cofactor is bounded by a quadratic function of the cutoff; therefore each fiber injects into a finite set of r -smooth integers, and the exponent-box estimate applies. Lean’s `proof_6_2` composes explicit finite-set/cardinality lemmas rather than appealing to an asymptotic black box. $\square$

What Lean strengthened in 6.2

The original report gave the polylogarithmic heuristic/proof in concise asymptotic form. The formal development exposes an explicit finite product `smoothBoxBound` and proves it as an actual cardinality upper bound before deriving the big-picture polylogarithmic estimate. This separation makes the dependence on r and on the anchor threshold precise.

6.3. Zero density: the new conditional formal proof

Define `divisorCount(o, a, r, x)` as the number of prime $q \leq x$ for which r divides the kernel, and `fiberCount(o, a, r, x)` as the number for which the output equals r . The fixed-anchor relative density is

$$\frac{\text{fiberCount}(o, a, r, x)}{\text{divisorCount}(o, a, r, x)}.$$

Define `pairCount(o, r, x)` using ordered prime pairs $p, q \leq x$. The global pair density is

$$\frac{\text{pairCount}(o, r, x)}{\pi(x)^2}.$$

Theorem 6.3 (6.3: zero density, formally conditional only on PNT-AP). *For every odd prime r :*

- (i) *for every prime anchor a , the multiplicative fixed-output fiber has relative density 0 whenever $a \neq r$;*
- (ii) *the additive fixed-output fiber has relative density 0 whenever $(a + 1) \not\equiv 0 \pmod{r}$;*
- (iii) *the ordered-pair densities for both $\boxplus$ and $\boxtimes$ tend to 0.*

All three clauses are compiler-verified from `PrimeAPInput` by `proof_6_3_from_PNT_AP`.

For fixed-anchor density, Theorem 6.2 gives a numerator $O((\log x)^{\pi(r)})$, while the exact progression reduction of Theorem 6.1, together with PNT-AP, gives a denominator asymptotic to a constant multiple of $x/\log x$. Hence the ratio is bounded by a constant times

$$\frac{(\log x)^{\pi(r)+1}}{x} \rightarrow 0.$$

Lean proves the log-power limit using mathlib asymptotic machinery and then uses squeeze arguments.

For pair density, the local counting module proves the concrete inequalities

$$N_r^{(+)}(x), N_r^{(\times)}(x) \leq \pi(x) S_r((x+1)^2).$$

Consequently

$$\frac{N_r^{(\star)}(x)}{\pi(x)^2} \leq \frac{S_r((x+1)^2)}{\pi(x)}.$$

Under `PrimeAPInput`, one fixed reduced progression already supplies an eventual lower bound $\pi(x) \gg x/\log x$, while Theorem 6.2 gives $S_r((x+1)^2) = O((\log x)^{\pi(r)})$. The quotient tends to zero.

Conditional does not mean unverified

The Lean kernel has checked the entire implication

$$\text{PrimeAPInput} \implies \text{Claims.t6_3}.$$

What remains absent from the pinned project is a proof term for `PrimeAPInput` itself. This is materially stronger than having only an informal proof sketch: every finite counting reduction and every analytic consequence after assuming PNT-AP has been checked.

Theorem 6.4 (6.4: unbounded sections). *For fixed prime input, the additive and multiplicative sections are unbounded; for fixed prime exponent, the exponential section is unbounded in the base. This theorem is unconditional in Lean. The construction uses congruence classes and Dirichlet's theorem on primes in arithmetic progressions, a theorem available in mathlib [mat].*

7. Prime dynamics

7.1. Fixed points of additive translations

Fix a prime anchor a and write $G_a(q) = a \boxplus q$.

Theorem 7.1 (7.1: additive fixed points). *For prime q ,*

$$G_a(q) = q \iff q \mid a + 1 \quad \text{and} \quad \frac{a + 1}{q} + 1 \text{ is } q\text{-smooth}.$$

Hence G_a has only finitely many prime fixed points, all among the prime divisors of $a + 1$.

The identity follows by writing $a + q + 1 = q((a + 1)/q + 1)$ once $q \mid a + 1$ and applying the exact fiber criterion.

Theorem 7.2 (7.2: multiplicative translations have no fixed points). *For primes a, q , $a \boxtimes q$ equals neither a nor q . In particular, the left translation $q \mapsto a \boxtimes q$ has no fixed point.*

Theorem 7.3 (7.3: the anchor-two additive dynamics). *For $G_2(q) = 2 \boxplus q$:*

- (a) *the unique prime fixed point is 3;*
- (b) *$2 \leftrightarrow 5$ is a two-cycle;*
- (c) *if $q + 3$ is a power of 2, then $G_2(q) = 2$, so the orbit enters the two-cycle in one step.*

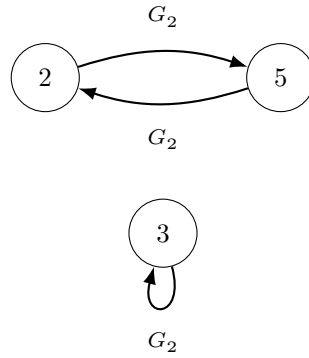


Figure 7.1.: The explicitly verified fixed point and two-cycle for the anchored additive map G_2 .

Theorem 7.4 (7.4: dual additive self-output). *For primes a, q ,*

$$a \boxplus q = a \iff a \mid q + 1 \quad \text{and} \quad \frac{q + 1}{a} + 1 \text{ is } a\text{-smooth}.$$

This is the symmetric counterpart of Theorem 7.1.

Theorem 7.5 (7.5: finite-state eventual periodicity). *Let $T : \mathbb{P} \rightarrow \mathbb{P}$ be any unary map obtained by fixing all but one input of a GPF operation. If the entire forward orbit of x lies in a finite set, then it is eventually periodic.*

This is simply the pigeonhole principle plus determinism: two orbit values repeat, and applying the same function thereafter forces repetition forever. Lean formalizes the general finite-state argument, not merely the special GPF case.

8. Exponential GPF structure and the Zsigmondy frontier

8.1. Unconditional elementary structure

Theorem 8.1 (8.1: no output two and nonsurjectivity). *For all primes p, q , $p \uparrow_G q$ is an odd prime. Consequently the map $(p, q) \mapsto p \uparrow_G q$ is not surjective onto the primes.*

The formal proof is direct. If $p = 2$, then $2^q + 1$ is odd and greater than 1, so its greatest prime factor is odd. If p is odd and $q = 2$, then $p^2 + 1 \equiv 2 \pmod{4}$ and is greater than 2, hence it is not a power of 2 and has an odd prime divisor. If p and q are both odd, the factorization of $p^q + 1$ contains an odd factor greater than 1. Thus the greatest prime factor is never 2. No Zsigmondy theorem is needed.

Theorem 8.2 (8.2: exponent-two congruence). *For every prime p ,*

$$p \uparrow_G 2 \equiv 1 \pmod{4}.$$

By the bridge identity of Theorem 9.1, this is the same value as $p \boxtimes p$.

8.2. Primitive divisors and Bang–Zsigmondy

Primitive prime divisors

A prime r is a primitive prime divisor of $a^n - b^n$ when it divides $a^n - b^n$ but no earlier $a^k - b^k$ with $1 \leq k < n$. In group language, when $r \nmid ab$, this means the residue a/b has exact multiplicative order n modulo r . Bang and Zsigmondy proved classical existence theorems with explicit exceptional cases [Ban86; Zsi92].

The formal project isolates exactly the specialized existence statement it still needs:

```
def ZsigmondyInput : Prop := p q,
  Nat.Prime p → Nat.Prime q → q > 2 → (p, q) ≠ (2, 3) →
  r, PrimitiveDivisor r p (2 * q) r p ^ q + 1
```

The intended classical route is transparent. Apply Bang–Zsigmondy to $p^{2q} - 1$. A primitive prime divisor r has order $2q$, so it cannot divide $p^q - 1$. Since

$$p^{2q} - 1 = (p^q - 1)(p^q + 1),$$

it must divide $p^q + 1$. The exceptional pair $(p, q) = (2, 3)$ corresponds to the classical $2^6 - 1$ exception; the condition $q \neq 2$ removes the low even-exponent situation relevant to the specialized statement.

Theorem 8.3 (8.3: Zsigmondy lower bound — conditional). *Let p, q be primes with $q \neq 2$ and $(p, q) \neq (2, 3)$. Assuming `ZsigmondyInput`, there is a prime $r \mid p^q + 1$ satisfying*

$$r \equiv 1 \pmod{2q}.$$

Consequently

$$p \uparrow_G q \geq 2q + 1.$$

The implication from the primitive divisor to the congruence and output bound is fully checked; only the existence input is external.

Theorem 8.4 (8.4: near anti-projection — conditional). *For primes p, q , $p \uparrow_G q \neq p$, and, assuming the same Zsigmondy input,*

$$p \uparrow_G q = q \iff (p, q) = (2, 3).$$

Theorem 8.5 (8.5: anchored exponential dynamics — conditional). *For a fixed prime anchor $a \neq 2$, the map $E_a(q) = a \uparrow_G q$ satisfies $E_a(q) > q$ for every prime q ; hence each forward orbit is strictly increasing, unbounded, and nonperiodic. For $a = 2$, 3 is fixed and every other prime is strictly increased; thus 3 is the only periodic point.*

Current formal boundary

Results 8.3–8.5 are not informal conjectures: Lean has checked their proofs from `ZsigmondyInput`. A single formal proof of that input would immediately turn all three into unconditional results and would also discharge Theorem 9.3.

9. Mixed-operation collisions

Theorem 9.1 (9.1: bridge identities). *For every prime p ,*

$$p \boxplus p = p \boxtimes 2, \quad p \uparrow_G 2 = p \boxtimes p.$$

These are kernel identities: $p + p + 1 = 2p + 1$ and $p^2 + 1 = pp + 1$.

Theorem 9.2 (9.2: second-column rigidity). *Let*

$$A_2(p) = p \boxplus 2, \quad M_2(p) = p \boxtimes 2, \quad E_2(p) = p \uparrow_G 2.$$

If any two are equal, their common value is 5.

The proof combines divisibility by the corresponding kernels to force the common prime to divide a small integer, then eliminates impossible factors.

Theorem 9.3 (9.3: exponential dominance on the $p = 2$ row — conditional). *For prime $q > 3$, assuming $ZsigmondyInput$,*

$$2 \boxplus q < 2 \uparrow_G q, \quad 2 \boxtimes q \leq 2 \uparrow_G q.$$

If $2q + 1$ is composite, the second inequality is strict.

The lower bound in Theorem 8.3 gives $2 \uparrow_G q \geq 2q + 1$, while greatest-prime-factor outputs are at most their kernels.

Theorem 9.4 (9.4: additive–multiplicative collision compatibility). *Fix primes p, r with $r \neq p$. The congruences*

$$r \mid p + q + 1, \quad r \mid pq + 1$$

have a simultaneous residue solution $q \pmod{r}$ iff

$$r \mid p^2 + p - 1.$$

When it exists, the residue is unique and

$$q \equiv -p - 1 \equiv -p^{-1} \pmod{r}.$$

Consequently, if $p \boxplus q = p \boxtimes q = r$, then $r \mid p^2 + p - 1$.

Eliminating q gives the quadratic polynomial: multiply the additive relation by p and subtract the multiplicative one.

Corollary 9.5 (9.5: finite common outputs). *For fixed prime p , every common output of $\boxplus$ and $\boxtimes$ is among the prime divisors of $p^2 + p - 1$. Hence there are only finitely many. The same finite containment holds for three-way collisions involving $\uparrow_G$.*

Theorem 9.6 (9.6: quadratic-residue restriction). *If prime r divides both $p + q + 1$ and $pq + 1$, then*

$$r \mid p^2 + p - 1, \quad r \mid q^2 + q - 1.$$

If additionally $r \neq 2, 5$, then 5 is a quadratic residue modulo r , and therefore

$$r \equiv 1 \text{ or } 4 \pmod{5}.$$

The discriminant of $X^2 + X - 1$ is 5. Existence of a root modulo r therefore yields a square root of 5; quadratic reciprocity then gives the residue restriction [IR90].

Theorem 9.7 (9.7: multiplicative–exponential obstruction). *If*

$$p \boxtimes q = p \uparrow_G q = r,$$

then

$$p^{q-1} \equiv q \pmod{r}.$$

Both kernels are -1 modulo r : $pq \equiv p^q \equiv -1$, and cancelling the invertible residue p yields the result.

Theorem 9.8 (9.8: three-way collision obstruction). *If*

$$p \boxplus q = p \boxtimes q = p \uparrow_G q = r,$$

then

$$r \mid p^2 + p - 1, \quad p^{q-1} \equiv q \pmod{r}, \quad q \equiv -p - 1 \pmod{r}.$$

If $r \neq 2, 5$, then $r \equiv \pm 1 \pmod{5}$.

This theorem is a clean composition of the independently verified collision results 9.4, 9.6, and 9.7.

10. Lean formalization and proof-status map

10.1. Architecture of the source

The project separates mathematical layers rather than placing all proofs in one file. The main modules are:

<code>Core.lean</code>	executable greatest-prime-factor scan; smoothness; operations; orbits; order predicates.
<code>Statements.lean</code>	proposition inventory for the numbered results.
<code>Elementary.lean</code>	elementary algebraic laws, exact fibers, fixed points, and historical refutation of old 5.5.
<code>Arithmetic.lean</code>	congruences, collision elimination, and mixed-operation arithmetic.
<code>SmallFibers.lean</code>	exact output-2 and output-3 classifications.
<code>Dynamics.lean</code>	generic eventual-periodicity argument.
<code>DirectExponential.lean</code>	unconditional odd-output structure for the exponential operation.
<code>Orders.lean</code>	finite-field orders, primitive-divisor bridges, 5.2, 5.3, 8.2, and the corrected 5.5 proof.
<code>FiveFive.lean</code>	official corrected numbered wrapper for 5.5.
<code>Quadratic.lean</code>	quadratic-character and quadratic-reciprocity results.
<code>Unbounded.lean</code>	unbounded-section theorem 6.4.
<code>Progressions.lean</code>	exact AP reductions, <code>PrimeAPInput</code> , and conditional 6.1.
<code>Counting.lean</code>	smooth-number and fiber bounds for 6.2; pair-count bounds for 6.3.
<code>Density.lean</code>	local analytic bridge proving 6.3 from <code>PrimeAPInput</code> .
<code>Conditional.lean</code>	<code>ZsigmondyInput</code> and its downstream wrappers.

10.2. Complete numbered-status table

Table 10.1: Formal status of the 39 corrected numbered results

Result	Topic	Status	Principal Lean declaration / module
3.1	Closure and divisibility	Verified	<code>proof_3_1</code> , Elementary
3.2	Commutativity split	Verified	<code>proof_3_2</code> , Elementary
3.3	Nonassociativity	Verified	<code>proof_3_3</code> , Elementary
3.4	Anti-projection / identities	Verified	<code>proof_3_4</code> , Elementary
3.5	Odd-prime closure	Verified	<code>proof_3_5</code> , DirectExponential
3.6	Cancellation / distributivity failures	Verified	<code>proof_3_6</code> , Elementary
3.7	Mixed associativity failures	Verified	<code>proof_3_7</code> , Elementary
3.8	Monotonicity failures	Verified	<code>proof_3_8</code> , Elementary
4.1	Exact fibers	Verified	<code>proof_4_1</code> , Elementary
4.2	Prime kernel and output two	Verified	<code>proof_4_2</code> , Elementary
4.3	Small fibers	Verified	<code>proof_4_3</code> , SmallFibers
5.1	Modular inverse laws	Verified	<code>proof_5_1</code> , Arithmetic
5.2	Diagonal order and congruence	Verified	<code>proof_5_2</code> , Orders
5.3	Diagonal orbit confinement	Verified	<code>proof_5_3</code> , Orders
5.4	Quadratic-character constraint	Verified	<code>proof_5_4</code> , Quadratic
5.5	Corrected dyadic forcing ($r \neq 2$)	Verified	<code>proof_5_5</code> , FiveFive / Orders
5.6	Parity split	Verified	<code>proof_5_6</code> , Arithmetic
6.1	AP reduction + PNT-AP	Conditional	<code>proof_6_1_from_PNT_AP</code> , Progressions; needs <code>PrimeAPIInput</code>
6.2	Polylogarithmic fibers	Verified	<code>proof_6_2</code> , Counting
6.3	Zero density	Conditional	<code>proof_6_3_from_PNT_AP</code> , Density; needs <code>PrimeAPIInput</code>
6.4	Unbounded sections	Verified	<code>proof_6_4</code> , Unbounded
7.1	Additive fixed points	Verified	<code>proof_7_1</code> , Elementary
7.2	Multiplicative fixed points	Verified	<code>proof_7_2</code> , Elementary
7.3	Anchor-two additive dynamics	Verified	<code>proof_7_3</code> , Elementary
7.4	Dual additive self-output	Verified	<code>proof_7_4</code> , Elementary
7.5	Finite-state eventual periodicity	Verified	<code>proof_7_5</code> , Dynamics
8.1	No exponential output two	Verified	<code>proof_8_1</code> , DirectExponential
8.2	Exponent-two congruence	Verified	<code>proof_8_2</code> , Orders

Continued on next page

Table 10.1: Formal status of the 39 corrected numbered results (Continued)

Result	Topic	Status	Principal Lean declaration / module
8.3	Zsigmondy lower bound	Conditional	<code>proof_8_3_from_zsigmondy</code> , Conditional; needs <code>ZsigmondyInput</code>
8.4	Near anti-projection	Conditional	<code>proof_8_4_from_zsigmondy</code> , Conditional
8.5	Exponential dynamics	Conditional	<code>proof_8_5_from_zsigmondy</code> , Conditional
9.1	Bridge identities	Verified	<code>proof_9_1</code> , Elementary
9.2	Second-column rigidity	Verified	<code>proof_9_2</code> , Arithmetic
9.3	Exponential dominance	Conditional	<code>proof_9_3_from_zsigmondy</code> , Conditional
9.4	Collision compatibility	Verified	<code>proof_9_4</code> , Arithmetic
9.5	Finite common outputs	Verified	<code>proof_9_5</code> , Arithmetic
9.6	Quadratic-residue restriction	Verified	<code>proof_9_6</code> , Quadratic
9.7	Multiplicative–exponential obstruction	Verified	<code>proof_9_7</code> , Arithmetic
9.8	Three-way collision obstruction	Verified	<code>proof_9_8</code> , Quadratic

10.3. What formalization clarified

The Lean development did more than encode existing prose. It exposed several points that are mathematically informative:

1. **Exceptional cases become unavoidable.** The old 5.5 failed precisely because characteristic two collapses -1 and 1 . Formal proof pressure isolated $r \neq 2$ as the missing hypothesis.
2. **Positivity assumptions on order matter.** The project’s raw primitive-divisor predicate admits the formal edge case $k = 0$; the generic order-divides- $r - 1$ statement is correct only for $k > 0$.
3. **Finite combinatorics separates cleanly from analytic input.** The exact residue-class reductions and the entire smooth/fiber-counting theory are unconditional. Only PNT-AP is external in 6.1 and 6.3.
4. **Zsigmondy is localized.** The order-theoretic consequences are checked; four numbered results depend on one existence proposition.
5. **Theorem 6.2 became more explicit.** The proof now contains a finite exponent-box bound rather than only asymptotic notation.

11. Related work and novelty assessment

11.1. Search methodology

A targeted prior-art search was conducted on 13 September 2026. Sources included ordinary web search, publisher/journal pages, Google-indexed scholarly records, OEIS, GitHub, current mathlib source/documentation, and searches for exact formula strings. Representative queries included:

- “greatest prime factor related magmas”, “prime magmas greatest prime factor”, “prime GPF magma”;
- exact or near-exact forms $\text{gpf}(p+q+1)$, $\text{gpf}(pq+1)$, $\text{gpf}(p^q+1)$, and variants using P^+ or “largest prime factor”;
- “Zsigmondy Lean”, “primitive divisor mathlib”, “prime number theorem arithmetic progression Lean mathlib”;
- conceptual searches on smooth-number fibers, prime dynamical systems, and greatest-prime-factor recurrences.

Negative search results are inherently non-exhaustive: absence from the returned corpus is not a proof of novelty.

11.2. Direct prior art on greatest-prime-factor magmas

The broad idea of defining magmas on primes via a greatest-prime-factor map is *not new*. Caragiu and Back introduced a family of infinite nonassociative magmas on the primes using the greatest prime factor and studied a particular commutative example in 2009 [CB09]. Caragiu and Vicol later studied the family

$$f_{a,b}(p, q) = P(ap + bq)$$

for positive integers a, b , including cyclicity questions [CV16]. Earlier and related papers study greatest-prime-factor sequences and recurrences [CS06; BC10].

This prior art materially changes how originality should be described. “Prime magmas defined using the greatest prime factor” cannot be claimed as an original concept of the present project.

11.3. Comparison with the three kernels in this report

The 2016 affine family $P(ap + bq)$ is structurally different from the multiplicative kernel $P(pq + 1)$ and the exponential kernel $P(p^q + 1)$. The additive kernel $P(p + q + 1)$ is affine with an added constant term and is not literally of the homogeneous form $P(ap + bq)$. The abstract of the 2009 paper does not disclose every operation in its family, so the search record does not justify claiming that no earlier paper contains an equivalent shifted operation.

Exact-formula and phrase searches performed for this revision did not locate an indexed scholarly publication directly matching the complete triple

$$P^+(p + q + 1), \quad P^+(pq + 1), \quad P^+(p^q + 1)$$

or the same theorem package (fibers, density, dynamics, and cross-operation collision restrictions). The correct conclusion is therefore limited:

Novelty assessment

Operations: greatest-prime-factor prime magmas have clear prior art. No directly matching publication for the exact three-kernel system was located in the searches performed, but this does not establish absolute novelty.

Individual theorems: many proof ingredients are classical; some statements appear tailored to these kernels and no direct antecedent was located, but a theorem-by-theorem exhaustive literature proof of novelty was not achieved.

Formalization: no directly matching public Lean formalization of this exact prime-GPF system was located. This supports a cautious claim of apparent formalization novelty, not a universal claim of first formalization.

11.4. Classical ingredients versus project-specific synthesis

The following ingredients are unquestionably classical: smooth-number theory [Dic30; Bru51; HT93]; primes in arithmetic progressions [Dav00]; multiplicative orders and quadratic reciprocity [IR90]; cyclotomic polynomials [Was97]; and primitive-divisor existence [Ban86; Zsi92]. Potential originality therefore lies, if anywhere, in the choice of kernels, the specific deductions linking them, the density/fiber questions posed for them, and the mechanized organization of those deductions.

11.5. Formal-library search

Current mathlib’s `PrimesInAP` module proves Dirichlet infinitude in reduced classes and provides theorems such as `Nat.infinite_setOfPred_prime_and_eq_mod` and `Nat.forall_exists_prime_gt_and_eq_mod` [mat]. These are infinitude statements, not the counting asymptotic required by `PrimeAPIInput`. A separate public Lean project, *PrimeNumberTheoremAnd*, explicitly lists the prime number theorem in arithmetic progressions among its formalization objectives [Kc]. This is relevant evidence of ongoing formal work, but it is not part of the PrimeGPF project’s pinned mathlib dependency and does not by itself discharge `PrimeAPIInput`.

The web/GitHub searches for this report did not locate a directly matching Bang–Zsigmondy existence theorem in current mathlib. Public Lean code outside mathlib does use custom primitive-prime-divisor predicates in other contexts, so the underlying formal concept is not unique to PrimeGPF; no source located in this search directly proves the specialized `ZsigmondyInput` used here. These are search findings, not assertions of nonexistence; specialized repositories, unindexed work, or later commits may change the picture.

12. Conclusions and open problems

The corrected project has reached a useful boundary: all elementary algebra, fixed-fiber structure, smooth counting, collision arithmetic, and finite dynamics in the numbered inventory are kernel-checked, while six results depend on only two mathematically natural external propositions.

12.1. Immediate formal targets

1. **Formalize PNT-AP in the needed form.** A proof of `PrimeAPIInput` would make 6.1 and 6.3 unconditional. Because the project’s finite residue/count identities are already proved, this task is essentially a bridge from a quantitative analytic-number-theory theorem to the exact `apCount/Tendsto` normalization.
2. **Formalize the specialized Zsigmondy existence statement.** A proof of `ZsigmondyInput` would simultaneously discharge 8.3, 8.4, 8.5, and 9.3.

12.2. Mathematical directions

Several questions suggested by the formal structure remain attractive independently of Lean:

- sharpen the polylogarithmic fixed- r fiber bounds and determine asymptotic constants where possible;
- classify exceptional small fibers beyond outputs 2 and 3;
- study cycle structure and escape rates of anchored translations, especially for $\uparrow_G$;
- quantify the rarity of double and triple collisions and exploit the polynomial divisor restrictions in 9.4–9.8;
- compare the exact three-kernel system systematically with the previously studied greatest-prime-factor magma families.

12.3. Interpretive conclusion

The greatest benefit of formalization here is not merely a “verified” label. It is the decomposition of the argument into exact finite arithmetic, explicit exceptional cases, and two clearly isolated classical inputs. That decomposition both corrected a false theorem and sharpened the mathematical roadmap.

A. Computational appendix: PARI/GP

The original report included exploratory PARI/GP routines. PARI/GP’s official documentation describes `factorint` and the arithmetic-function factoring machinery [PAR]. The following compact version reproduces the basic greatest-prime-factor operations. It is useful for experimentation but is not a substitute for formal proof.

Listing A.1: Basic PARI/GP definitions for the three GPF operations

```
greatestPrimeFactor(n) = {
  my(F = factor(n));
  F[matsize(F)[1], 1]
};

gpfpPlus(p,q) = greatestPrimeFactor(p + q + 1);
gpfpTimes(p,q) = greatestPrimeFactor(p*q + 1);
gpfpPower(p,q) = greatestPrimeFactor(p^q + 1);

primePrefix(k) = vector(k, i, prime(i));

gpfpPowerTable(k) = {
  my(P = primePrefix(k));
  matrix(k,k,i,j,gpfpPower(P[i],P[j]))
};

fiberTimes(r,B) = {
  my(P=listcreate(), out=List());
  forprime(p=2,B,listput(P,p));
  for(i=1,#P, for(j=1,#P,
    if(gpfpTimes(P[i],P[j])==r, listput(out,[P[i],P[j]))
  ));
  Vec(out)
};
```

For large inputs, care is needed: the documentation notes that factorization routines may use probable-prime stages depending on settings. For theorem discovery, computational data are valuable; for theorem certification, the Lean development is the stronger artifact.

B. Lean declaration guide

Table B.1:

Formal object	Mathematical role
<code>gpfn</code>	executable greatest-prime-factor function
<code>IsGPFnr</code>	r is prime, divides n , and dominates all prime divisors of n
<code>Smoothrn</code>	positive r -smooth integer
<code>kernelopq</code>	one of $p + q + 1$, $pq + 1$, $p^q + 1$
<code>outputopq</code>	greatest prime factor of the kernel
<code>Prime</code>	subtype of natural numbers carrying a primality proof
<code>orbitfxn</code>	n th iterate of f on x
<code>ExactOrderprk</code>	explicit least positive exponent condition modulo r
<code>PrimitiveDivisorrk</code>	prime r realizing exact exponent k in project notation
<code>primeCountx</code>	primes $\leq x$
<code>apCountrcx</code>	primes $\leq x$ in a residue class
<code>divisorCountoarx</code>	prime inputs whose kernel is divisible by r
<code>fiberCountoarx</code>	prime inputs whose output equals r
<code>pairCountorx</code>	ordered prime pairs $\leq x$ with output r
<code>smoothCountrx</code>	positive r -smooth integers $\leq x$
<code>APAsymptoticrc</code>	PNT-AP ratio expressed as a filter limit
<code>RelativeZerooar</code>	fixed-anchor fiber/divisibility ratio tends to 0
<code>PairZeroor</code>	global ordered-pair density tends to 0

C. Complete AI-use disclosure

Scope of AI assistance

This report was produced with substantial AI assistance. AI/deep-research and browsing tools were used to locate candidate literature, compare bibliographic records, synthesize mathematical background, organize the exposition, draft and edit prose, generate XeLaTeX/TikZ/listing source, and assist with document quality control. The user supplied the mathematical project, source compendium, Lean project context, build/audit transcripts, and direction for the revision.

What AI assistance does not establish

AI-generated prose is not a proof. Web search is not an exhaustive literature review. A plausible citation can be wrong or incomplete. A mathematical explanation can mistranscribe a hypothesis. Therefore the report distinguishes three evidential levels:

1. **Lean-verified claims:** status is based on supplied successful build/audit evidence and source declarations. These claims are about formal propositions actually accepted by Lean, subject to the correctness of the formal statement and imported environment.
2. **Source-derived mathematical context:** classical facts are attributed to books, papers, or official documentation and should be checked against those sources for publication use.
3. **Research synthesis and novelty assessment:** conclusions about prior art are inferences from a targeted, non-exhaustive search. Negative results mean only that no direct match was located in the searches performed.

Formal verification is independent of AI validation

The Lean proof checker does not accept a theorem because an AI judges it correct. Tactics and generated scripts must elaborate into proof terms accepted by Lean's kernel. The project's audit additionally checks for disallowed escape hatches. Lean's documentation explains this trust model and also stresses that kernel acceptance does not by itself prove that the human-readable interpretation of a formal statement is the intended one [Leaa; Leab].

Recommended human review before publication

A human author or referee should: (i) compare every displayed theorem against the current committed Lean source; (ii) rerun the pinned Lean build and axiom audit; (iii) verify the bibliographic metadata and page/DOI data against primary sources; (iv) examine the novelty-search log and repeat searches in MathSciNet/zbMATH/Google Scholar where access permits; (v) check that the informal proof sketches faithfully represent the formal argument; and (vi) decide what originality language is appropriate for the intended venue.

D. Change log and author-review checklist

Major mathematical corrections

- Replaced the old false numbered Theorem 5.5 by the formally verified corrected theorem with $r \neq 2$; retained the counterexample as historical evidence.
- Upgraded Theorem 6.2 to unconditional compiler-verified status and documented the explicit exponent-box proof.
- Upgraded Theorem 6.3 from an informal density argument to a compiler-verified implication from `PrimeAPIInput`, including pair-count and limit architecture.
- Marked 6.1, 6.3, 8.3, 8.4, 8.5, and 9.3 explicitly conditional rather than presenting their external analytic/primitive-divisor inputs as already formalized.

Explanatory additions

Added introductions to magmas, greatest prime factors, fibers, smooth numbers, residue rings, multiplicative orders, primitive divisors, cyclotomic polynomials, quadratic characters, PNT-AP, filter limits, density notions, and finite dynamics, together with a source-module guide and formal proof-status table.

Literature and novelty additions

Added classical smooth-number, PNT-AP, cyclotomic, reciprocity, and Bang–Zsigmondy references; identified direct prior art on greatest-prime-factor prime magmas; and added a dated, cautious novelty assessment for the exact three-kernel system and its Lean formalization.

Items requiring author review

- Push the newest local `Density.lean`/coverage milestone to GitHub so remote and local source records agree.
- Before making a publication-level originality claim, repeat the novelty search in MathSciNet and zbMATH.
- Decide whether the historical old 5.5 should remain in the main narrative or be moved entirely to an erratum appendix.
- Confirm preferred notation for the exponential operation; this edition uses $\uparrow_G$ to avoid fragile glyphs.

Bibliography

- [Ban86] A. S. Bang. “Talttheoretiske Undersøgelser”. In: *Tidsskrift for Mathematik*. 5th ser. 4 (1886), pp. 70–80, 130–137. URL: <https://www.jstor.org/stable/i24539913> (visited on 09/13/2026).
- [BC10] G. Back and M. Caragiu. “The Greatest Prime Factor and Recurrent Sequences”. In: *The Fibonacci Quarterly* 48.4 (2010), pp. 358–362. DOI: [10.1080/00150517.2010.12428090](https://doi.org/10.1080/00150517.2010.12428090). URL: https://fq.math.ca/Papers1/48-4/Back_Caragiu.pdf.
- [Bru51] N. G. de Bruijn. “On the Number of Positive Integers not Exceeding x and Free of Prime Factors Greater than y ”. In: *Indagationes Mathematicae* 13 (1951), pp. 50–60.
- [BS81] S. Burris and H. P. Sankappanavar. *A Course in Universal Algebra*. Graduate Texts in Mathematics 78. New York: Springer-Verlag, 1981.
- [CB09] M. Caragiu and G. Back. “The Greatest Prime Factor and Related Magmas”. In: *JP Journal of Algebra, Number Theory and Applications* 15.2 (2009), pp. 127–136. URL: https://www.pphmj.com/article.php?act=art_abstract_show&art_id=4569&flag=prev (visited on 09/13/2026).
- [CS06] M. Caragiu and L. Scheckelhoff. “The Greatest Prime Factor and Related Sequences”. In: *JP Journal of Algebra, Number Theory and Applications* 6.2 (2006), pp. 403–409. URL: <https://www.pphmj.com/journals/articles/281.htm> (visited on 09/13/2026).
- [CV16] M. Caragiu and P. A. Vicol. “Prime Magmas and a Cyclicity Conjecture”. In: *JP Journal of Algebra, Number Theory and Applications* 38.2 (2016), pp. 129–143. DOI: [10.17654/NT038020129](https://doi.org/10.17654/NT038020129). URL: <https://www.pphmj.com/abstract/9685.htm>.
- [Dav00] H. Davenport. *Multiplicative Number Theory*. 3rd ed. Graduate Texts in Mathematics 74. New York: Springer, 2000.
- [Dic30] K. Dickman. “On the Frequency of Numbers Containing Prime Factors of a Certain Relative Magnitude”. In: *Arkiv för Matematik, Astronomi och Fysik* 22A.10 (1930), pp. 1–14.
- [Hel26] A. Hellström. *Theorem Compendium for Prime GPF Magmas*. Project compendium dated 17 June 2026; source document supplied for this revision. June 2026.
- [HT86] A. Hildebrand and G. Tenenbaum. “On Integers Free of Large Prime Factors”. In: *Transactions of the American Mathematical Society* 296.1 (1986), pp. 265–290. DOI: [10.1090/S0002-9947-1986-0837811-1](https://doi.org/10.1090/S0002-9947-1986-0837811-1).
- [HT93] A. Hildebrand and G. Tenenbaum. “Integers without Large Prime Factors”. In: *Journal de Théorie des Nombres de Bordeaux* 5.2 (1993), pp. 411–484. DOI: [10.5802/jtnb.101](https://doi.org/10.5802/jtnb.101). URL: <https://jtnb.centre-mersenne.org/articles/10.5802/jtnb.101/>.
- [IR90] K. Ireland and M. Rosen. *A Classical Introduction to Modern Number Theory*. 2nd ed. Graduate Texts in Mathematics 84. New York: Springer, 1990. DOI: [10.1007/978-1-4757-2103-4](https://doi.org/10.1007/978-1-4757-2103-4).
- [Kc] A. Kontorovich and contributors. *PrimeNumberTheoremAnd: Blueprint for the PNT+ Project*. Public Lean project whose stated objectives include the prime number theorem in arithmetic progressions. URL: <https://github.com/AlexKontorovich/PrimeNumberTheoremAnd> (visited on 09/13/2026).
- [Leaa] Lean FRO. *Frequently Asked Questions — Lean*. URL: <https://lean-lang.org/faq/> (visited on 09/13/2026).
- [Leab] Lean FRO. *Validating a Lean Proof*. URL: <https://lean-lang.org/doc/reference/latest/ValidatingProofs/> (visited on 09/13/2026).
- [mat] mathlib contributors. *Mathlib.NumberTheory.LSeries.PrimesInAP*. URL: https://leanprover-community.github.io/mathlib4_docs/Mathlib/NumberTheory/LSeries/PrimesInAP.html (visited on 09/13/2026).

- [OEI] OEIS Foundation Inc. *OEIS A006530: Greatest Prime Factor of n* . URL: <https://oeis.org/A006530> (visited on 09/13/2026).
- [PAR] PARI/GP Development Team. *Catalogue of GP/PARI Functions: Arithmetic Functions*. URL: https://pari.math.u-bordeaux.fr/dochtm/html-stable/Arithmetic_functions.html (visited on 09/13/2026).
- [Was97] L. C. Washington. *Introduction to Cyclotomic Fields*. 2nd ed. Graduate Texts in Mathematics 83. New York: Springer, 1997. DOI: [10.1007/978-1-4612-1934-7](https://doi.org/10.1007/978-1-4612-1934-7).
- [Wei] E. W. Weisstein. *Greatest Prime Factor*. MathWorld—A Wolfram Web Resource. URL: <https://mathworld.wolfram.com/GreatestPrimeFactor.html> (visited on 09/13/2026).
- [Zsi92] K. Zsigmondy. “Zur Theorie der Potenzreste”. In: *Monatshefte für Mathematik und Physik* 3 (1892), pp. 265–284. DOI: [10.1007/BF01692444](https://doi.org/10.1007/BF01692444).